
REQUIREMENTS NOT MET

- N/A

VIDEO FILE LINK

https://youtu.be/9NDKh_dyTkW?si=J35rxo81L2_9OJpW

PROBLEMS ENCOUNTERED

- N/A

FUTURE WORK/APPLICATIONS

The biggest thing that this final lab shows is how to start using CPU's to be more functional, while originally we had to choose mux selects and manually put our inputs with a DAD board, we then added a controller, making it much easier to do a function, and finally we have taken it a step further by allowing EPROM and SRAM to control our inputs, which has taken the process of selecting multiplexers and then input codes to a simple chart of machine codes, allowing s to hand assemble a program.

This lab was actually kind of sick.

PRE-LAB QUESTIONS OR EXERCISES

```
% File name = eprom.mif                                     %
% ***** %
% Sample program for G-CPU                                   %
% ***** %

DEPTH = 4096;  % Address Bus Size %
WIDTH = 8;      % Data Format %

ADDRESS_RADIX = HEX;  % Address Format %
DATA_RADIX = HEX;      % Data Format %

CONTENT
BEGIN

0      :      08;  %      LDX    #$1800      %
1      :      00;  %      Little Endian 16-bit data 00      %
2      :      18;  %      Big Endian 16-bit data 18      %

3      :      09;  %      LDY    #$1900 %
4      :      00;  %      Little Endian 16-bit data 00      %
5      :      19;  %      Big Endian 16-bit data 19      %

6      :      0C;  %      (LOOP start) LDAA 0,X      %
7      :      00;  %      looks at value stored in X + 0      %

8      :      00;  %      TAB      %

9      :      13;  %      STAB 0,Y      %
A      :      00;  %      dd = 0      %
% Store data in B to memory location pointed to by Y (indexed addressing) %

B      :      30;  %      INX      %
C      :      31;  %      INY      %

D      :      21;  %      BNE LOOP      %
E      :      06;

[F..FFF] :      00;  % zero rest of memory %
END;
```

Figure 1: .mif file for Part 1 EPROM.

In our EPROM, we are doing the following:

1. Loading the value \$1800 to X
2. Loading the value \$1900 to Y
3. Starting a loop
4. Loads A with the value in address X + 0 (to X)
5. Stores the value in A to B
6. Stores the value of B to the address Y + 0 (to Y)
7. Increments X & Y, loops if A != 0.

```
% File name = sram.mif                                     %  
% ***** %  
  
DEPTH = 4096;      % Address Bus Size %  
WIDTH = 8;         % Data Bus Size %  
  
ADDRESS_RADIX      = HEX;  % Address Format %  
DATA_RADIX         = HEX;  % Data Format %  
  
CONTENT  
BEGIN  
[0..7FF]           :      FF;      % zero memory %  
800                :      02;  
801                :      37;  
802                :      00;  
803                :      9D;  
804                :      9D;  
805                :      02;  
806                :      02;  
807                :      02;  
808                :      02;  
809                :      02;  
80A                :      02;  
80B                :      00;  
[80C..FFF]         :      00;  
  
END;
```

Figure 2: Given SRAM for Part 1.

Table 1: Example Run of Given EPROM and SRAM

Address(es) [S]	Opcodes [S]	Instruction	A [S]	B [S]	X [S]	Y [S]	Z	N	PC [S]	Comments
0000- 0002	08 00 18	LDX #\$1800	00	00	1800	0000	1	0	0003	Loads 1800 to X
0003- 0005	09 00 19	LDY #\$1900	00	00	1800	1900	1	0	0006	Loads 1900 to Y
0006- 0007	0C 00	LDAA \$00, X	02	00	1800	1900	0	0	0008	Loads value in SRAM at \$800 (02) to A
0008	00	TAB	02	02	1800	1900	0	0	0009	Loads value in A to B
0009- 000A	13 00	STAB \$00, Y	02	02	1800	1900	0	0	000B	Stores register B value to Y at register \$1900
000B	30	INX	02	02	1801	1900	0	0	000C	Increments X
000C	31	INY	02	02	1801	1901	0	0	000D	Increments Y
000D- 000E	21 06	BNE \$06	02	02	1801	1901	0	0	0006	Checks if Z = 1, loops if not.
0006- 0007	0C 00	LDAA \$00, X	37	02	1801	1901	0	0	0008	Loads value in SRAM at \$801 (37) to A
0008	00	TAB	37	37	1801	1901	0	0	0009	Loads value in A to B
0009- 000A	13 00	STAB \$00, Y	37	37	1801	1901	0	0	000B	Stores register B value to Y at register \$1901
000B	30	INX	37	37	1802	1901	0	0	000C	Increments X
000C	31	INY	37	37	1802	1902	0	0	000D	Increments Y
000D- 000E	21 06	BNE \$06	37	37	1802	1902	0	0	0006	Checks if Z = 1, loops if not.
0006- 0007	0C 00	LDAA \$00, X	00	37	1802	1902	1	0	0008	Loads value in SRAM at \$802 (00) to A
0008	00	TAB	00	00	1802	1902	1	0	0009	Loads value in A to B
0009- 000A	13 00	STAB \$00, Y	00	00	1802	1902	1	0	000B	Stores register B value to Y at register \$1902
000B	30	INX	00	00	1802	1902	1	0	000C	Increments X
000C	31	INY	00	00	1803	1903	1	0	000D	Increments Y
000D- 000E	21 06	BNE \$06	00	00	1803	1902	1	0	000F	Checks if Z = 1, branching to 000F (00 for rest)

We can see above that our two flags are Z and N, Z is true when $A \neq 0$, and N is true for when A is a negative value. Z starts true but quickly is turned off when we load our data, only becoming true towards the end of our run, which causes the branch to happen (ending the loop.)

Our pattern is as follows: Load value to X/Y, Load value in RAM address X to A, Load $A \rightarrow B$, Store B to Register in Y, Increment X, Increment Y, Loop back to Load value in RAM Address X to A if $A \neq 0$.

Test Runs (ONE FULL LOOP EACH):

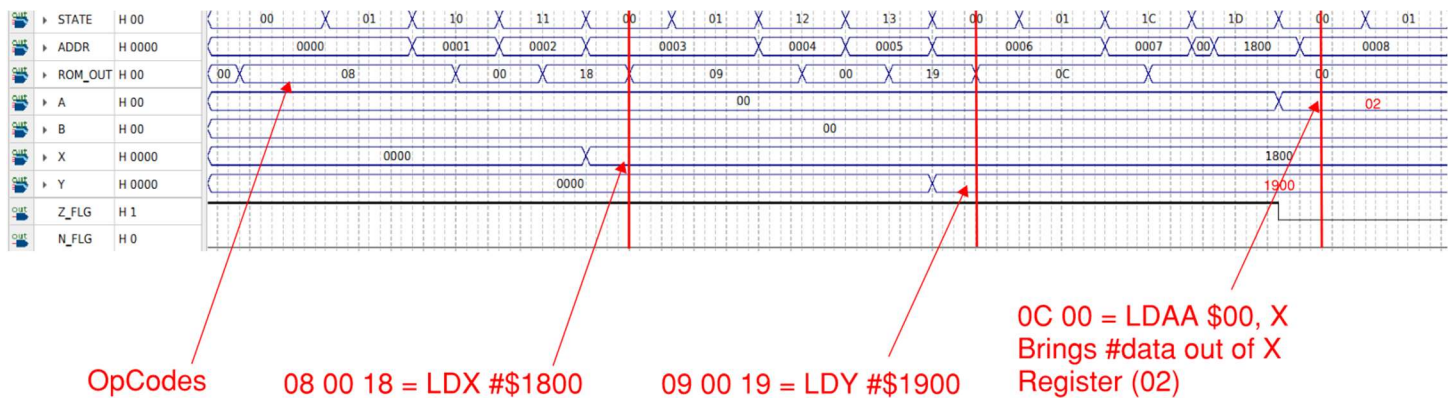


Figure 3: Prefilled EPROM Test Run, one full loop. (PT. 1)

Additional to above notes: We see that as soon as $A \neq 0$, $Z_FLG \rightarrow 0$. We can also see that the negative flag never fires as we do not have any negative numbers.

End of Last

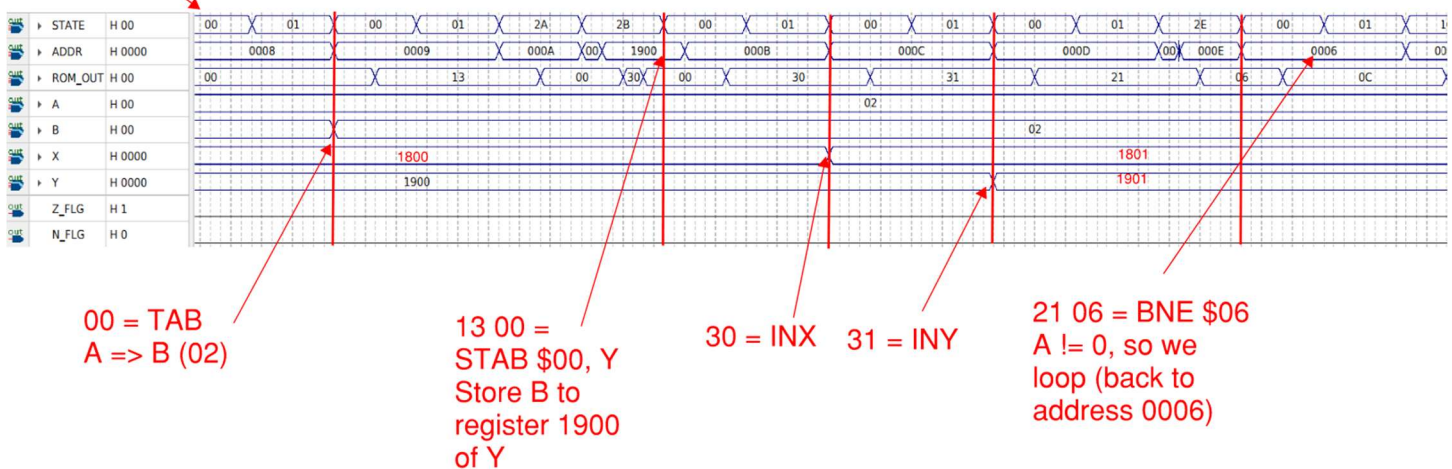


Figure 4: Prefilled EPROM Test Run, one full loop. (PT. 2)

```
% File name = sram.mif %  
% ***** %  
  
DEPTH = 4096; % Address Bus Size %  
WIDTH = 8; % Data Bus Size %  
  
ADDRESS_RADIX = HEX; % Address Format %  
DATA_RADIX = HEX; % Data Format %  
  
CONTENT  
BEGIN  
[0..7FF] : FF; % zero memory %  
800 : 07;  
801 : 20;  
802 : 00;  
803 : 9D;  
804 : 9D;  
805 : 02;  
806 : 02;  
807 : 02;  
808 : 02;  
809 : 02;  
80A : 02;  
80B : 00;  
[80C..FFF] : 00;  
  
END;
```

Figure 5: Edited SRAM for Part 1.

Table 2: Example Run of Edited EPROM and SRAM

Address(es) [S]	Opcodes [S]	Instruction	A [S]	B [S]	X [S]	Y [S]	Z	N	PC [S]	Comments
0000- 0002	08 00 18	LDX #\$1800	00	00	1800	0000	1	0	0003	Loads 1800 to X
0003- 0005	09 00 19	LDY #\$1900	00	00	1800	1900	1	0	0006	Loads 1900 to Y
0006- 0007	0C 00	LDAA \$00, X	07	00	1800	1900	0	0	0008	Loads value in SRAM at \$800 (07) to A
0008	00	TAB	07	07	1800	1900	0	0	0009	Loads value in A to B
0009- 000A	13 00	STAB \$00, Y	07	07	1800	1900	0	0	000B	Stores register B value to Y at register \$1900
000B	30	INX	07	07	1801	1900	0	0	000C	Increments X
000C	31	INY	07	07	1801	1901	0	0	000D	Increments Y
000D- 000E	21 06	BNE \$06	07	07	1801	1901	0	0	0006	Checks if Z = 1, loops if not.
0006- 0007	0C 00	LDAA \$00, X	20	07	1801	1901	0	0	0008	Loads value in SRAM at \$801 (20) to A
0008	00	TAB	20	20	1801	1901	0	0	0009	Loads value in A to B
0009- 000A	13 00	STAB \$00, Y	20	20	1801	1901	0	0	000B	Stores register B value to Y at register \$1901
000B	30	INX	20	20	1802	1901	0	0	000C	Increments X
000C	31	INY	20	20	1802	1902	0	0	000D	Increments Y
000D- 000E	21 06	BNE \$06	20	20	1802	1902	0	0	0006	Checks if Z = 1, loops if not.
0006- 0007	0C 00	LDAA \$00, X	00	20	1802	1902	1	0	0008	Loads value in SRAM at \$802 (00) to A
0008	00	TAB	00	00	1802	1902	1	0	0009	Loads value in A to B
0009- 000A	13 00	STAB \$00, Y	00	00	1802	1902	1	0	000B	Stores register B value to Y at register \$1902
000B	30	INX	00	00	1802	1902	1	0	000C	Increments X
000C	31	INY	00	00	1803	1903	1	0	000D	Increments Y
000D- 000E	21 06	BNE \$06	00	00	1803	1902	1	0	000F	Checks if Z = 1, branching to 000F (00 for rest)

Like before, we see that Z only becomes true for when A = 0, and N stays false as we do not have any negative numbers in A. The only thing that changes are the values which are loaded for each register. I have kept 0 in register 803 to keep similarity between the two runs.

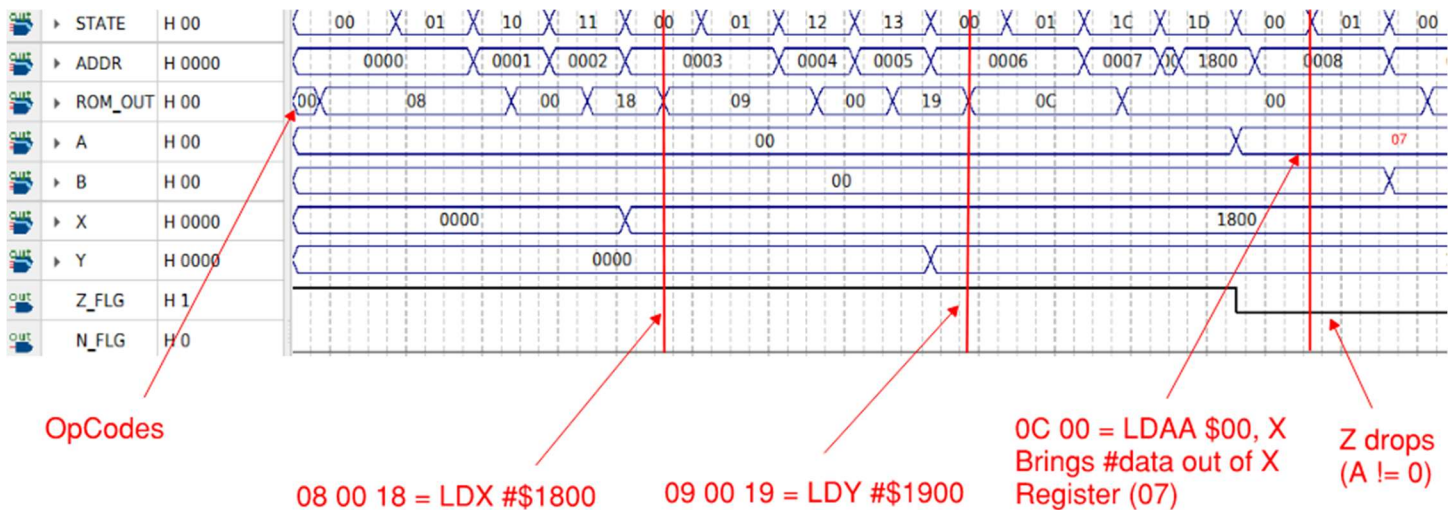


Figure 6: Edited EPROM Test Run, one full loop. (PT. 1)

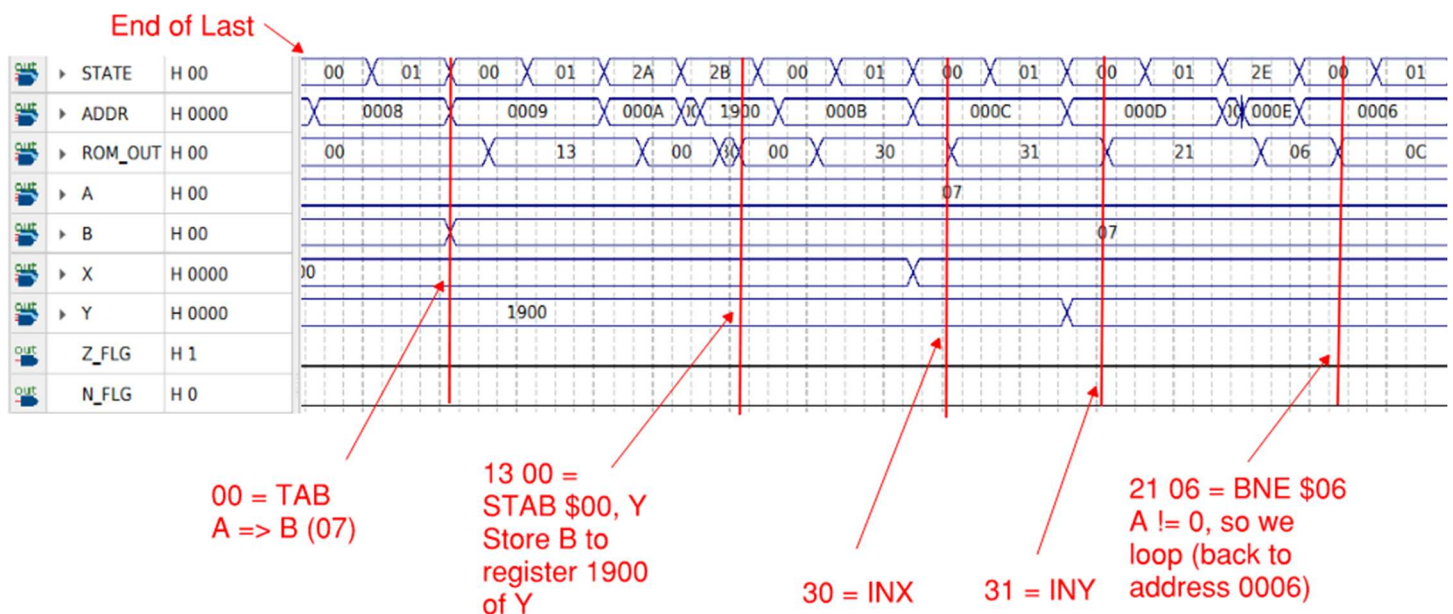


Figure 7: Edited EPROM Test Run, one full loop. (PT. 2)

Here we can see our program still follows the same pattern, but we input different numbers; the SRAM controls the data in/out and the EPROM controls the operation. We can still see that we load the same addresses in X and Y, we load data into A, copy it to B, store the data from B to register 1900 of Y (or the current value of Y for future cycles), increment X & Y, then loop if A != 0.

PART TWO

Table 3: $A - B \rightarrow Y$ | Subtraction Loop Program

Address(es) [\$]	Opcodes [\$]	Instruction	Comments
0000-0002	08 4B 1E	LDX #data	Using SRAM to load 1E4B, data to X (table val 1)
0003-0005	0B 4B 1E	LDY data	Using SRAM to load 1E4B, data register to Y
0006-0007	0C 02	LDAA 02, X	Loads A with data from register X + 2 (Tab Size)
0008-000A	06 00 10	STAA LoopTrack	Stores Tab size to LoopTrack, so we know when to stop.
000B-000C	02 01	LDAA #01	Loads A with the value of 1.
000D-000E	0E 04	LDAB 04, X	Loads B with the data at X+4, our first subtraction term.
000F	1B	COMB	Takes the compliment of B.
0010	15	SUM AB	Adds 1 to the compliment of B (two's comp.), store in B.
0011-0012	0C 03	LDAA 03, X	Loads A with the data at X+3, our first addition term.
0013	15	SUM AB	Adds A & B ($A + (-B)$), outputs to B
0014-0015	13 00	STAB 00, Y	Stores A to Y
0016	31	INY	Increments Y by 1 (one output term for two inputs)
0017	30	INX	Increments X by 1.
0018	30	INX	Increments X by 1.
0019-001B	04 00 10	LDAA LoopTrack	Loads A with our term for counting remaining loops.
001C-001D	03 FF	LDAB #FF	Loads negative 1 two's comp to B.
001E	14	SUM BA	Sums A + B, outputs to A. (LoopTrack - 1)
001F-0021	06 00 10	STAA LoopTrack	Stores new LoopTrack to A
0022-0023	21 0B	BNE Loop	If A != 0, we continue our loop.
0024-0025	20 24	BEQ Done	If A = 0, we break/branch.

Table 4: Hand-assembled Subtraction Program

Address(es) [S]	Opcodes [S]	Instruction	A [S]	B [S]	X [S]	Y [S]	Z	N	PC [S]	Comments
0000-0002	08 4B 1E	LDX #data	00	00	1E4B	0000	1	0	0003	1E4B data to X
0003-0005	0B 4B 1E	LDY data	00	00	1E4B	1A57	1	0	0006	1E4B register to Y
0006-0007	0C 02	LDAA 02, X	03	00	1E4B	1A57	0	0	0008	Loads A with tab size
0008-000A	06 00 10	STAA LoopTrack	03	00	1E4B	1A57	0	0	0009	Stores tab size to LoopTrack
000B-000C	02 01	LDAA #01	01	00	1E4B	1A57	0	0	000B	Loads A with the value of 1.
000D-000E	0E 04	LDAB 04, X	01	0F	1E4B	1A57	0	0	000C	Loads B with first subtraction term.
000F	1B	COMB	01	F0	1E4B	1A57	0	0	000D	Takes the compliment of B.
0010	15	SUM AB	01	F1	1E4B	1A57	0	0	0006	Stores two's comp in B.
0011-0012	0C 03	LDAA 03, X	FF	F1	1E4B	1A57	0	0	0008	Loads A with first addition term.
0013	15	SUM AB	FF	F0	1E4B	1A57	0	0	0009	Outputs two's comp to B.
0014-0015	13 00	STAB 00, Y	FF	F0	1E4B	1A57	0	0	000B	Stores B to Y.
0016	31	INY	FF	F0	1E4B	1A58	0	0	000C	Increments Y by 1.
0017	30	INX	FF	F0	1E4C	1A58	0	0	000D	Increments X by 1.
0018	30	INX	FF	F0	1E4D	1A58	0	0	0006	Increments X by 1.
0019-001B	04 00 10	LDAA LoopTrack	03	F0	1E4D	1A58	1	0	0008	Loads A with LoopTrack.
001C-001D	03 FF	LDAB #FF	03	FF	1E4D	1A58	1	0	0009	Loads -1's 2-comp to B.
001E	14	SUM BA	02	FF	1E4D	1A58	1	0	000B	Finds Looptrack - 1
001F-0021	06 00 10	STAA LoopTrack	02	FF	1E4D	1A58	1	0	000C	Stores LoopTrack to A
0022-0023	21 0B	BNE Loop	02	FF	1E4D	1A58	1	0	000D	Loop (A != 0)
000B-000C	02 01	LDAA #01	01	FF	1E4D	1A58	0	0	000B	Loads A with 1.
000D-000E	0E 04	LDAB 04, X	01	03	1E4D	1A58	0	0	000C	Loads B with second subtraction term.
000F	1B	COMB	01	FC	1E4D	1A58	0	0	000D	Takes the compliment of B.
0010	15	SUM AB	01	FD	1E4D	1A58	0	0	0006	Stores two's comp in B.
0011-0012	0C 03	LDAA 03, X	0A	FD	1E4D	1A58	0	0	0008	Loads A with second addition term.
0013	15	SUM AB	0A	07	1E4D	1A58	0	0	0009	Stores two's comp in B.
0014-0015	13 00	STAB 00, Y	0A	07	1E4D	1A58	0	0	000B	Stores B to Y.
0016	31	INY	0A	07	1E4D	1A59	0	0	000C	Increments Y by 1.
0017	30	INX	0A	07	1E4E	1A59	0	0	000D	Increments X by 1.
0018	30	INX	0A	07	1E4F	1A59	0	0	0006	Increments X by 1.

0019-001B	04 00 10	LDAA LoopTrack	02	07	1E4F	1A59	1	0	0008	Loads A with LoopTrack.
001C-001D	03 FF	LDAB #FF	02	FF	1E4F	1A59	1	0	0009	Loads -1 2's comp to B.
001E	14	SUM BA	01	FF	1E4F	1A59	1	0	000B	Finds Looptrack - 1
001F-0021	06 00 10	STAA LoopTrack	01	FF	1E4F	1A59	1	0	000C	Stores new LoopTrack
0022-0023	21 0B	BNE Loop	01	FF	1E4F	1A59	1	0	000D	Loop (A != 0)
000B-000C	02 01	LDAA #01	01	FF	1E4F	1A59	0	0	000B	Loads A = 1
000D-000E	0E 04	LDAB 04, X	01	04	1E4F	1A59	0	0	000C	Loads B with third subtraction term.
000F	1B	COMB	01	FB	1E4F	1A59	0	0	000D	Takes the compliment of B.
0010	15	SUM AB	01	FC	1E4F	1A59	0	0	0006	Stores two's comp in B.
0011-0012	0C 03	LDAA 03, X	05	FC	1E4F	1A59	0	0	0008	Loads A with third addition term.
0013	15	SUM AB	05	01	1E4F	1A59	0	0	0009	Stores two's comp in B.
0014-0015	13 00	STAB 00, Y	05	01	1E4F	1A59	0	0	000B	Stores B to Y.
0016	31	INY	05	01	1E4F	1A5A	0	0	000C	Increments Y by 1.
0017	30	INX	05	01	1E50	1A59	0	0	000D	Increments X by 1.
0018	30	INX	05	01	1E51	1A59	0	0	0006	Increments X by 1.
0019-001B	04 00 10	LDAA LoopTrack	01	01	1E51	1A59	1	0	0008	Loads A with LoopTrack.
001C-001D	03 FF	LDAB #FF	01	FF	1E51	1A59	1	0	0009	Loads -1 2's comp to B.
001E	14	SUM BA	00	FF	1E51	1A59	1	0	000B	Finds Looptrack - 1
001F-0021	06 00 10	STAA LoopTrack	00	FF	1E51	1A59	1	0	000C	Stores new LoopTrack
0022-0023	21 0B	BNE Loop	00	FF	1E51	1A59	1	0	000D	A = 0, no loop.
0024-0025	20 24	BEQ Done	00	FF	1E51	1A59	1	0	000F	If A = 0, we break/branch.

```
1  NegOne          equ      $FF
2  One             equ      $0001
3  Zero            equ      $0000
4
5  LoopTrack       equ      $1000
6  TableAddress    equ      $1E4B
7
8
9                  org      TableAddress
10 TableData       dc.b     $57, $1A, $03
11                dc.b     $FF, $0F
12                dc.b     $0A, 03
13                dc.b     $05, 04
```

Figure 8: G-IDE Assembly Part 1

15		org	Zero
16		LDX	#TableData
17		LDY	TableAddress
18		LDAA	02, X
19		STAA	LoopTrack
20	Loop	LDAA	#One
21		LDAB	04, X
22		COMB	
23		SUM_AB	
24		LDAA	03, X
25		SUM_AB	
26		STAB	00, Y
27		INY	
28		INX	
29		INX	
30		LDAA	LoopTrack
31		LDAB	#NegOne

Figure 9: G-IDE Assembly Part 2

32		SUM_BA	
33		STAA	LoopTrack
34		BNE	Loop
35	Done	BEQ	Done

Figure 10: G-IDE Assembly Part 3

```
% Automated build for Lab7.asm %
DEPTH = 4096;
WIDTH = 8;
ADDRESS_RADIX = HEX;
DATA_RADIX = HEX;
CONTENT
BEGIN

[0000..0e4a]      :      00;

0e4b              :      57;      -- TableData dc.b $57,$1A,$03
0e4c              :      1A;
0e4d              :      03;

0e4e              :      FF;      -- dc.b $FF,$0F
0e4f              :      0F;

0e50              :      0A;      -- dc.b $0A,03
0e51              :      03;

0e52              :      05;      -- dc.b $05,04
0e53              :      04;

[0e54..0fff]      :      00;

END
```

Figure 11: RAM .mif

EEL 3701C: Digital Logic & Computer Systems
Revision 0
Lab 7 Report: Assembly Programming

```
% Automated build for Lab7.asm %
DEPTH = 4096;
WIDTH = 8;
ADDRESS_RADIX = HEX;
DATA_RADIX = HEX;
CONTENT
BEGIN

0000      :      08;    -- LDX #TableData
0001      :      48;
0002      :      1E;

0003      :      0B;    -- LDY TableAddress
0004      :      4B;
0005      :      1E;

0006      :      0C;    -- LDAA 02,X
0007      :      02;

0008      :      06;    -- STAA LoopTrack
0009      :      00;
000a      :      10;

000b      :      02;    -- LDAA #One
000c      :      01;

000d      :      0E;    -- LDAB 04,X
000e      :      04;

000f      :      1B;    -- COMB
0010      :      15;    -- SUM_AB
0011      :      0C;    -- LDAA 03,X
0012      :      03;

0013      :      15;    -- SUM_AB
0014      :      13;    -- STAB 00,Y
0015      :      00;

0016      :      31;    -- INY
0017      :      30;    -- INX
0018      :      30;    -- INX
0019      :      04;    -- LDAA LoopTrack
001a      :      00;
001b      :      10;

001c      :      03;    -- LDAB #NegOne
001d      :      FF;

001e      :      14;    -- SUM_BA
001f      :      06;    -- STAA LoopTrack
0020      :      00;
0021      :      10;

0022      :      21;    -- BNE Loop
0023      :      0B;

0024      :      20;    -- BEQ Done
0025      :      24;

[0026..0fff] :      00;

END
```

Figure 12: ROM .mif

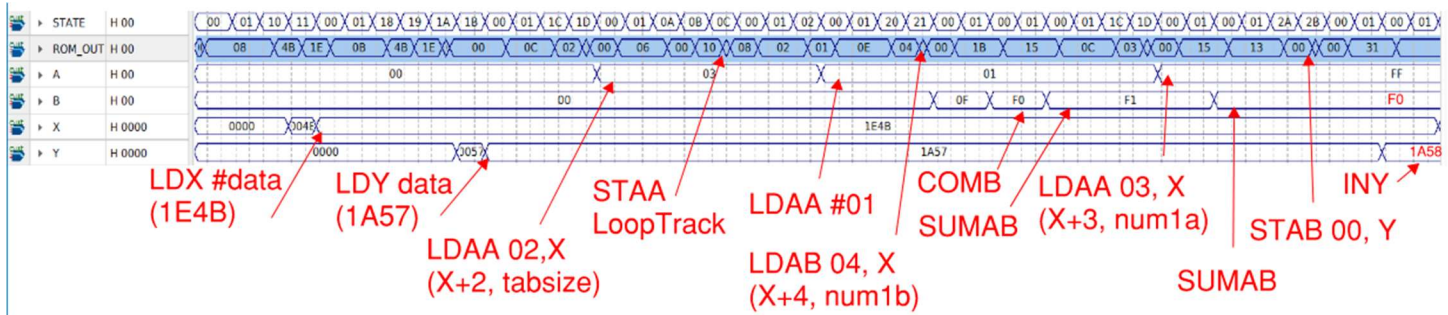


Figure 13: Quartus Waveform for EPROM and SRAM (Pt. 1)

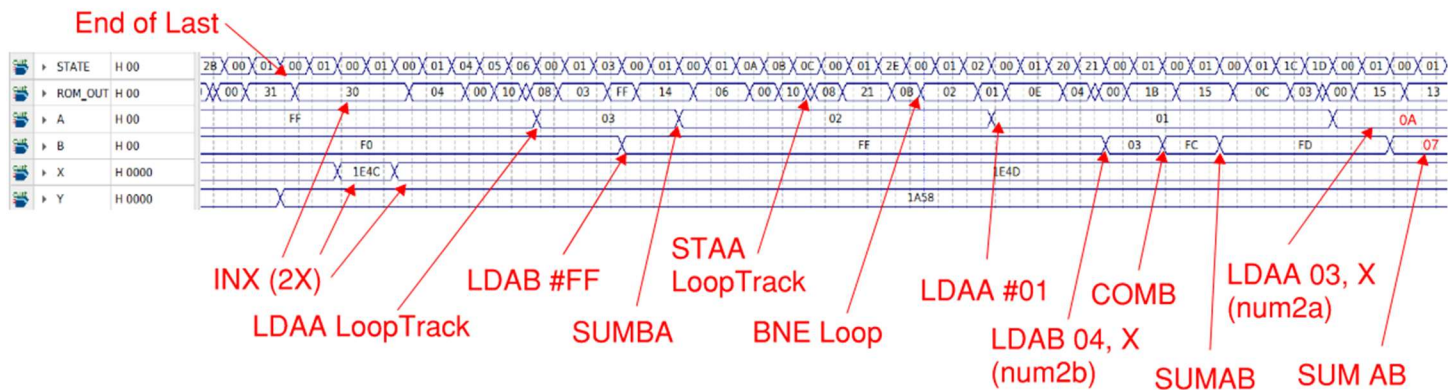


Figure 14: Quartus Waveform for EPROM and SRAM (Pt. 2)

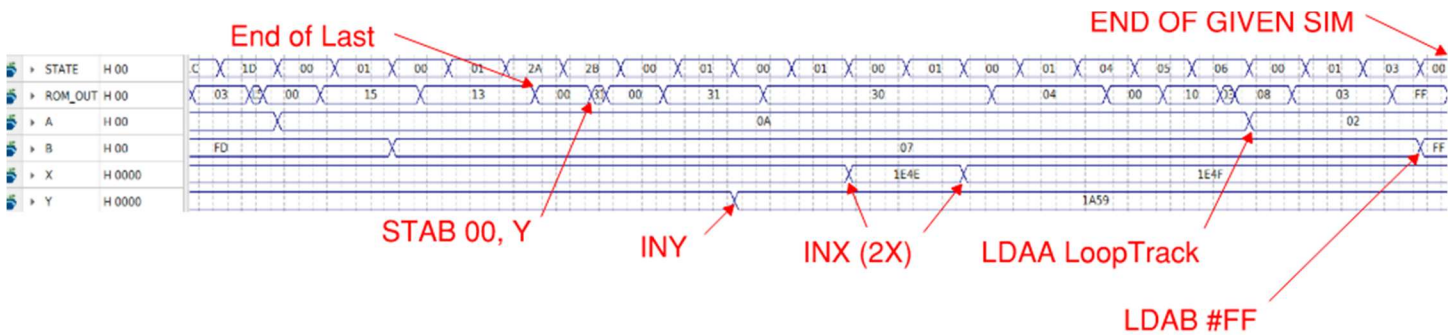


Figure 15: Quartus Waveform for EPROM and SRAM (Pt. 3)

Additional Annotations on next page.

Annotations

For our data, we can see we follow our pattern from the table:

1. We load X with our data address (X -> 1E4B)
2. We load the data in the address to Y (Y -> 1A57)
3. We load X with the data offset by 2, which is our third value in the table: TabSize, this keeps track of how many times we will be looping, we will decrement after each successful loop.
4. STAA LoopTrack stores the value in A to the variable LoopTrack
5. LDAA #01 loads A with 1, so that we can make the twos complement of B
6. LDAB 04, X takes num1b from the table and loads it to B.
7. COMB takes the complement of B
8. SUMAB adds 1 to the complement, completing the two's complement of B.
9. LDAA 03, X then loads A with num1a, so that we can do our subtraction.
10. SUMAB does the addition of A + -B
11. STAB 00, Y stores the output to the current Y register.
12. INY increments Y
13. INX increments X, we do this twice as we have numXa and numXb, so we need two input inc. per output inc.
14. LDAA LoopTrack loads LoopTrack to A.
15. LDAB #FF loads the twos complement of 1 so we can decrement LoopTrack
16. SUMBA decrements LoopTrack and stores it to A
17. STAA LoopTrack stores the decremented LoopTrack
18. BNE Loop loops when LoopTrack != 0
19. LDAA #01 loads 1 to A
20. LDAB 04, X loads num2b to B
21. COMB finds the complement of B
22. SUMAB completes the two's complement of B and stores it to B
23. LDAA 03, X loads num2a and stores it to A
24. SUM AB completes term two subtraction.
25. STAB 00, Y stores the second output to the new Y (original Y + 1)
26. INY increments Y
27. INX x2 increments X twice
28. LDAA LoopTrack loads looptrack to A
29. LDAB #FF loads the two's complement of 1 to B.
30. Given simulation ends at 20 us.